

Habitudes d'écriture et tests unitaires

Nommer, commenter, décomposer, factoriser — puis vérifier

Sommaire

PARTIE I · HABITUDES D'ÉCRITURE

01	Pourquoi le clean code	3
02	Nommer	3
03	Commenter	4
04	Décomposer	5
05	Factoriser	7

PARTIE II · TESTS UNITAIRES

06	Le cadre et l'objectif	9
07	Les jeux de tests	9
08	Test en boîte noire	10
09	Test en boîte blanche	11
10	Mise en œuvre : l'instruction assert	12
11	Compléments et pièges	14

POUR RÉVISER

·	Glossaire des définitions	15
·	Synthèse	15

01 Pourquoi le *clean code*

diapos 3–4

OBJECTIFS DU CHAPITRE

- sensibiliser à quelques dispositifs qui permettent d'**améliorer la qualité d'un code** (*Clean code*), afin qu'il réponde mieux aux attentes de ceux à qui il s'adresse : relecteurs, collaborateurs, clients... ;
- sensibiliser à la **nécessité professionnelle de « bien écrire »**.

La définition à retenir est économique, pas esthétique :

DÉFINITION

L'objectif principal du *Clean code* est la **construction de logiciels en minimisant le coût du changement** (évolution du besoin, changement de l'équipe qui gère...) **et en maximisant la capacité à répondre au changement**.

Autrement dit : on n'écrit pas proprement pour faire joli, mais parce qu'un logiciel est **modifié** bien plus souvent qu'il n'est écrit, et que chaque modification coûte d'autant plus cher que le code est difficile à comprendre.

Les quatre pistes de travail

PISTE	CE QU'ELLE DEMANDE
Nommer	les variables et fonctions selon des conventions propres au contexte
Commenter	le code pour l' expliquer ou le structurer
Décomposer	en fonctions , fonctions auxiliaires et code principal
Factoriser	les fonctionnalités redondantes

Deux autres pistes

- **KISS** — *Keep It Simple, Stupid* : éviter l'**optimisation prématurée**. On écrit d'abord un code simple et correct ; on n'optimise que si une mesure montre que c'est nécessaire.
- **La règle du boy scout** : **laisser le code source plus propre que quand on l'a trouvé**. Une petite amélioration à chaque passage, plutôt qu'un grand nettoyage qui n'arrive jamais.

02 Nommer

diapos 5–8

Le cours ouvre cette partie sur une carte de France du *pain au chocolat* et de la *chocolatine*, et sur une citation d'Ambroise de Milan (IV^e siècle) : *Si fueris Romae, Romano vivito more; si fueris alibi, vivito sicut ibi* — « **À Rome, fais comme les Romains !** »

PRINCIPE GÉNÉRAL

- les conventions **peuvent changer** selon le langage, l'entreprise, les personnes avec qui on travaille ;
- la compétence visée est donc de **s'adapter** et de les utiliser **selon le contexte**.

Il n'y a donc pas de « bonne » convention dans l'absolu : ce qui suit décrit celles utilisées dans l'UE **Info1.Algo1**.

Les conventions de l'UE

- en **français**, pour la bonne compréhension ;
- utiliser des **noms précis** ;
- un nom de **variable** reflète **ce que représente** la variable ;
- un nom de **fonction** reflète **ce que fait** la fonction — **le quoi, pas le comment** — si possible avec un **verbe d'action** ;
- refléter le **niveau d'abstraction** auquel on travaille : un nom fonctionnel au niveau fonctionnel, un nom technique au niveau technique.

PLUS PRÉCISÉMENT

RÈGLE	EXEMPLE
préconisations Python : snake_case (voir PEP 8, <i>Style Guide for Python Code</i>)	<code>liste_valeurs</code>
variables et fonctions booléennes : commencent par <code>est_...</code> ou <code>a_...</code>	<code>est_palindrome</code> , <code>a_doublons</code>
distinguer une valeur et son indice dans un tableau ou une liste	<code>valeur</code> / <code>i_valeur</code>
listes : le pluriel peut suffire	<code>liste_valeurs</code> ou <code>valeurs</code>
variables de comptage	<code>nb_elements</code>

Le test pratique du bon nom : si vous devez lire le corps d'une fonction pour savoir ce qu'elle fait, son nom est à revoir. `traiter(1)` ne dit rien ; `valeurs_superieures_a_la_moyenne(valeurs)` dit tout — et rend le commentaire inutile, ce qui est l'objet de la section suivante.

03 Commenter

diapos 9-10

Commenter pour expliquer

- en cours de développement : indiquer l'**intention** pour les lignes suivantes (*quitte à supprimer ensuite*) ;
- certains commentaires — tous ? — **peuvent être évités quand le nommage est bien fait** ;
- indiquer **le pourquoi**, et **pas le comment**.

À FAIRE ON EXPLIQUE LE POURQUOI

```
# Division entière pour éviter les flottants
moyenne = somme//nb_elements
```

À ÉVITER ON PARAPHRASE LE COMMENT

```
# Division de la somme par le nombre d'éléments
moyenne = somme//nb_elements
```

Le second commentaire ne fait que redire la ligne en français : il ajoute du texte à maintenir sans ajouter d'information. Le premier explique un choix que le code seul ne révèle pas — pourquoi // plutôt que /.

Commenter pour structurer

Sur un **code long** (par exemple au-delà de 1000 lignes), les commentaires servent à **identifier la structure du code** :

- **titres et séparations** entre les parties fonctionnelles ;
- ne pas dépasser les **80 caractères par ligne**.

STRUCTURE TYPE

```
# FONCTIONS ENTREES-SORTIES
def ...

# FONCTIONS UTILITAIRES
def ...

# TRAITEMENT PRINCIPAL
def ...

# PROGRAMME PRINCIPAL
...
```

Cette structure n'est pas décorative : elle annonce exactement la décomposition de la section suivante.

04 Décomposer

diapos 11-15

AVANTAGES DE LA DÉCOMPOSITION EN FONCTIONS

- **Lisibilité** ;
- **Modularité et réutilisabilité**, dans le même ou dans d'autres programmes ;
- **Testabilité et maintenance** : tests unitaires, diagnostic d'erreurs, mises à jour.

Le troisième point fait le lien avec la partie II : **on ne peut tester unitairement que ce qui a été isolé dans une fonction**. Décomposer n'est donc pas une coquetterie de style, c'est la condition d'existence des tests.

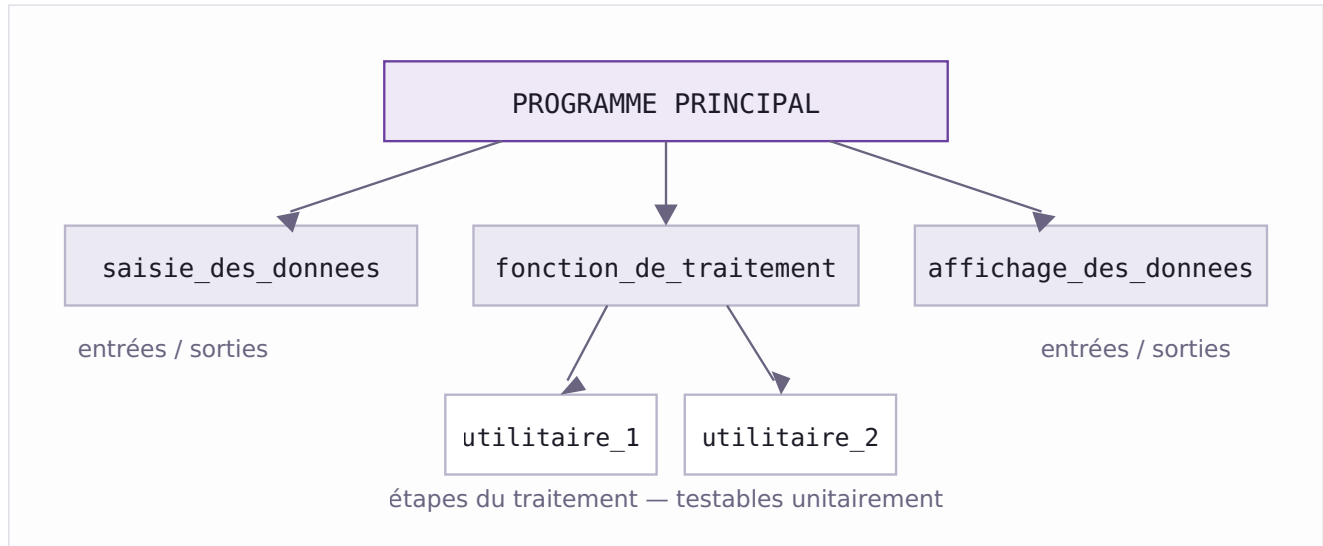
La méthode : de haut en bas

On commence par le **programme principal** :

```
# PROGRAMME PRINCIPAL
entrees = saisie_des_donnees()
sorties = fonction_de_traitement(entrees)
affichage_des_donnees(sorties)
```

Puis on remonte en précisant les **étapes du traitement** :

```
def fonction_de_traitement(entrees):
    fonction_utilitaire_1(...)
    fonction_utilitaire_2(...)
    return ...
```



On écrit d'abord le niveau le plus abstrait — **quoi** — puis on descend vers le **comment**. Les fonctions du bas ne font ni saisie ni affichage : ce sont exactement celles que la partie II saura tester.

L'exemple du cours

Le code suivant permet à l'utilisateur de saisir une liste d'entiers dont on calcule la moyenne, puis constitue et affiche la liste des nombres supérieurs ou égaux à cette moyenne. Il s'agit de le **décomposer en fonctions afin d'en améliorer la lisibilité**.

AVANT 16 LIGNES, TOUT AU MÊME NIVEAU

```
nb_elements = int(input("Nombre d'elements a saisir : "))
liste = []
for _ in range(nb_elements):
    valeur = int(input("Saisir un entier : "))
    liste.append(valeur)
somme = 0
for valeur in liste:
    somme += valeur
moyenne = somme/nb_elements
liste_superieurs = []
for valeur in liste:
    if valeur >= moyenne:
        liste_superieurs.append(valeur)
for valeur in liste_superieurs:
    print(valeur, end=' ')
print()
```

PROPOSITION DE CORRECTION

Une décomposition possible, suivant la structure en quatre sections vue plus haut. Le programme principal se lit maintenant en trois lignes.

```
# FONCTIONS ENTREES-SORTIES
def saisie_des_valeurs():
    nb_elements = int(input("Nombre d'elements a saisir : "))
    valeurs = []
    for _ in range(nb_elements):
        valeurs.append(int(input("Saisir un entier : ")))
    return valeurs

def affichage_des_valeurs(valeurs):
    for valeur in valeurs:
        print(valeur, end=' ')
    print()

# FONCTIONS UTILITAIRES
def moyenne(valeurs):
    somme = 0
    for valeur in valeurs:
        somme += valeur
    return somme / len(valeurs)

# TRAITEMENT PRINCIPAL
def valeurs_superieures_a_la_moyenne(valeurs):
    moyenne_des_valeurs = moyenne(valeurs)
    superieures = []
    for valeur in valeurs:
        if valeur >= moyenne_des_valeurs:
            superieures.append(valeur)
    return superieures

# PROGRAMME PRINCIPAL
valeurs = saisie_des_valeurs()
superieures = valeurs_superieures_a_la_moyenne(valeurs)
affichage_des_valeurs(superieures)
```

Ce qu'on a gagné, concrètement : `moyenne` et `valeurs_superieures_a_la_moyenne` ne contiennent **ni saisie ni affichage**. Elles prennent des paramètres et renvoient un résultat : ce sont des fonctions **testables**, au sens de la partie II. Au passage, `moyenne` est réutilisable ailleurs.

05 Factoriser

diapos 16-17

PRINCIPE GÉNÉRAL

- **repérer** des fonctions dont le code est très ressemblant ;
- **mettre en commun** (factoriser) les parties de code identiques ;
- les différences amènent à rajouter **un ou plusieurs paramètres supplémentaires**.

ATTENTION

Copier-coller du code → produire du code presque identique → code factorisable. Le copier-coller n'est pas interdit : il est un **signal** qu'il y aura quelque chose à factoriser juste après.

L'exemple du cours

On considère un point mobile repéré par ses coordonnées (x, y) dans un repère. Il s'agit de factoriser les trois fonctions `monter`, `descendre` et `droite` qui permettent de déplacer ce point.

AVANT TROIS FOIS LA MÊME MÉCANIQUE

```
def monter(point):
    x, y = point
    return (x, y+1)

def descendre(point):
    x, y = point
    return (x, y-1)

def droite(point):
    x, y = point
    return (x+1, y)
```

PROPOSITION DE CORRECTION

La partie identique est « décomposer le point, puis renvoyer un point translaté ». Les différences sont les deux valeurs ajoutées à x et à y : elles deviennent **deux paramètres supplémentaires**.

```
def deplacer(point, dx, dy):
    # dx et dy : déplacement demandé sur chaque axe
    x, y = point
    return (x + dx, y + dy)
```

Les trois fonctions d'origine peuvent alors disparaître au profit d'appels directs — `deplacer(point, 0, 1)`, `deplacer(point, 0, -1)`, `deplacer(point, 1, 0)` — ou être conservées comme noms de commodité, ce qui garde un vocabulaire lisible au niveau d'abstraction supérieur :

```
def monter(point):
    return deplacer(point, 0, 1)

def descendre(point):
    return deplacer(point, 0, -1)

def droite(point):
    return deplacer(point, 1, 0)

def gauche(point):
    return deplacer(point, -1, 0)
```

Le bénéfice se voit à la dernière ligne : ajouter `gauche` coûte désormais une ligne au lieu de trois, et une correction de bug dans le déplacement se fait à **un seul endroit**. C'est très exactement « minimiser le coût du changement ».

06 Le cadre et l'objectif

diapo 19

CADRE DU PROBLÈME

Le cadre d'étude proposé est celui des **fonctions**. Il est cependant **généralisable** à tout code présentant une ou plusieurs entrées, et une ou plusieurs sorties : programme principal, portion de code.

On **exclut** deux situations :

- les **interactions avec l'utilisateur** : dans le cadre étudié, les entrées sont données simultanément et les sorties recueillies simultanément ;
- les **effets de bord non testables** : affichage, etc.

OBJECTIF

Étant donnée une fonction, on souhaite répondre à la question suivante : **la fonction effectue-t-elle bien ce qu'on attend qu'elle fasse ?**

Ces exclusions expliquent rétrospectivement la section 04 : une fonction qui fait un `input` ou un `print` au milieu de son traitement est hors du cadre. C'est pourquoi la décomposition sépare les entrées-sorties du traitement.

07 Les jeux de tests

diapos 20–22

VOCABULAIRE

Un **jeu de test** permet de **spécifier**, pour une fonction et des **paramètres donnés**, la **valeur de retour attendue**.

MÉTHODE

Afin de vérifier si une fonction répond au problème posé :

- tester cette fonction sur des jeux de tests couvrant le **maximum de cas possibles** ;
- trouver les cas **particuliers**, les cas **extrêmes**, les cas **généraux**.

Objectif : mettre en défaut le code à tester.

Cette dernière phrase est le renversement mental à opérer. On n'écrit pas des tests pour se rassurer, mais pour **essayer de casser** sa propre fonction. Un test qui passe du premier coup sur le cas le plus banal n'apprend presque rien ; c'est la liste vide, le singleton ou la valeur limite qui révèle les bugs.

On étudie ainsi la **couverture de tests**, les **domaines des paramètres** et des **valeurs de retour** — étude qui se poursuivra lors de l'étude de la **spécification de fonction**.

DEUX FAÇONS DE FAIRE

En pratique, la méthodologie d'élaboration des tests peut s'avérer ardue ; elle sera vue plus en détail dans l'UE **Info2.ILU1**. Dans l'UE Info1.Algo1, on pourra vous demander d'élaborer des jeux de test de deux façons différentes :

- **sans connaître le code** de la fonction à tester — *test en boîte noire* : les cas à traiter seront donnés explicitement ;
- **en vous donnant le code** de la fonction à tester — *test en boîte blanche* : les cas seront caractérisés par la façon dont la fonction les exécute.

La fonction servant d'exemple

Rappel : on dit qu'une liste est un **palindrome** si elle se lit de la même manière dans les deux sens.

On considère la fonction `est_palindrome` qui accepte en paramètre une liste d'entiers et retourne le booléen indiquant si cette liste est (`True`) ou non (`False`) un palindrome. C'est sur elle que portent les quatre questions du cours, traitées dans les sections suivantes.

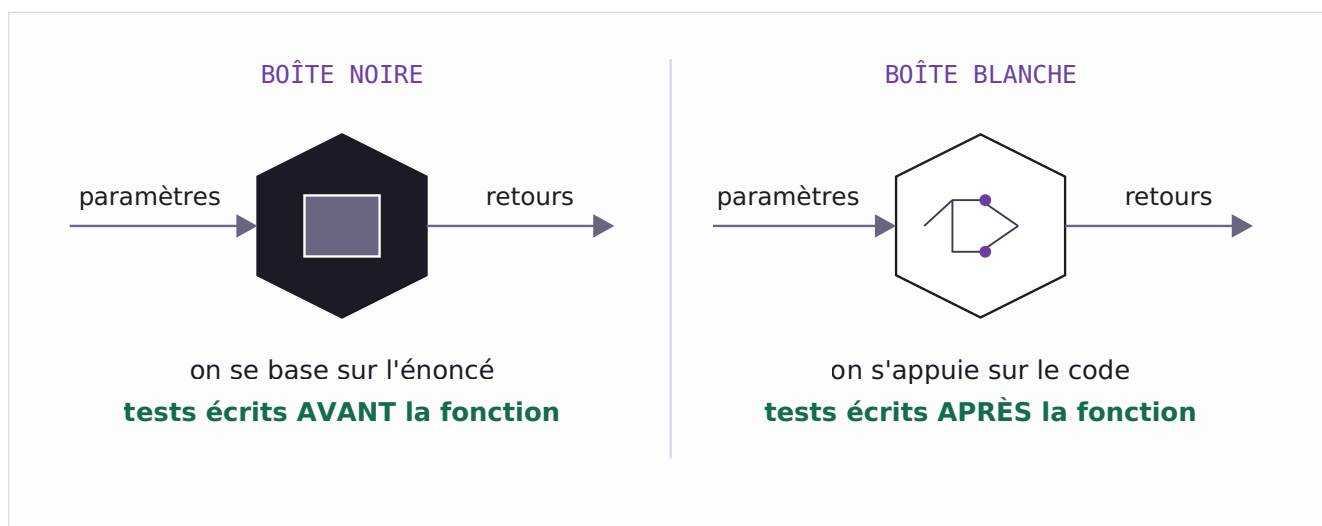
08 Test en boîte noire

diapos 23-24

DÉFINITION

Dans la méthode de test en **boîte noire**, on ne connaît pas le code de la fonction.

- on se base **uniquement** sur l'énoncé pour écrire les tests ;
- on peut donc écrire les tests **avant d'écrire la fonction** ;
- on cherche à couvrir l'**intégralité des besoins identifiés dans l'énoncé**.



La différence n'est pas la difficulté, c'est l'**information dont on dispose** — et donc le moment où l'on écrit les tests. En boîte blanche, un jeu de tests qui couvre l'**intégralité des lignes** d'une fonction est dit **complet**.

Question 2 du cours : six jeux de tests en boîte noire

L'énoncé demande six jeux de tests distincts respectant les caractéristiques suivantes : liste vide ; singleton ; liste de longueur impaire (pour les deux valeurs de retour possibles) ; liste de longueur paire (pour les deux valeurs de retour possibles).

PROPOSITION DE CORRECTION

CAS	PARAMÈTRE	RETOUR ATTENDU
liste vide	[]	True
singleton	[7]	True
longueur impaire, palindrome	[1, 2, 1]	True
longueur impaire, non palindrome	[1, 2, 3]	False
longueur paire, palindrome	[1, 2, 2, 1]	True
longueur paire, non palindrome	[1, 2, 3, 4]	False

Les deux premiers sont les **cas extrêmes** : ce sont eux qui piègent le plus souvent les implémentations. Une liste vide et un singleton sont des palindromes — ils se lisent identiquement dans les deux sens.

09 Test en boîte blanche

diapos 25-26

DÉFINITION

Dans la méthode de test en **boîte blanche**, on connaît le code de la fonction.

- on **s'appuie sur le code** pour identifier les cas à tester ;
- on doit donc écrire les tests **après avoir écrit la fonction** ;
- un jeu de tests qui **couvre l'intégralité des lignes** d'une fonction est dit **complet**.

Voici l'implémentation donnée par le cours :

IMPLÉMENTATION FOURNIE

```
def est_palindrome(liste):
    i, j = 0, len(liste)-1
    while i < j:
        if liste[i] != liste[j]:
            return False
        i += 1
        j -= 1
    return True
```

Deux indices se déplacent l'un vers l'autre depuis les extrémités. Les cas à tester ne se décrivent plus par la forme de la liste, mais par **ce que fait la boucle**.

Question 3 du cours : cinq jeux de tests en boîte blanche

PROPOSITION DE CORRECTION

CAS D'EXÉCUTION	PARA-MÈTRE	CE QUI SE PASSE	RE-TOUR
aucune itération	[]	$i=0, j=-1$: la condition $i < j$ est fausse d'emblée	True
une itération incomplète	[1, 2]	on entre dans la boucle et on sort par <code>return False</code> avant la fin du tour	False
exactement une itération	[1, 1]	un tour complet, puis $i=1, j=0$ arrête la boucle	True
une itération complète puis une incomplète	[1, 2, 3, 1]	premier tour concluant, sortie par <code>return False</code> au deuxième	False
exactement deux itérations complètes	[1, 2, 2, 1]	deux tours complets, puis $i=2, j=1$ arrête la boucle	True

Le singleton [7] convient également pour le premier cas ($i=0, j=0$), et [1, 2, 1] pour le troisième.

Comparez les deux tableaux : plusieurs jeux se ressemblent, mais **la justification change**. En boîte noire, [1, 2, 2, 1] est « une liste paire qui est un palindrome » ; en boîte blanche, c'est « le cas qui fait deux tours complets ». C'est cette justification qui est demandée en examen.

10 Mise en œuvre : l'instruction assert

diapos 27-29

PRINCIPES POUR LES TESTS

- **éviter les interactions utilisateur** (saisie, affichage) exigeant une vérification humaine entre l'obtenu et l'attendu ;
- **automatiser** les tests ;
- **regrouper les tests** dans une fonction que l'on peut décider d'appeler ou non.

DÉFINITION

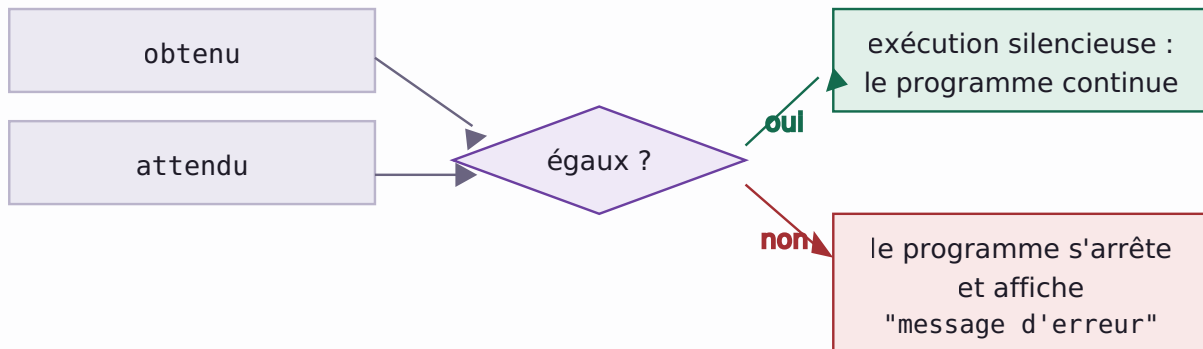
Une **assertion** est une **expression booléenne qui doit être évaluée à vrai**. Si l'évaluation de l'assertion est évaluée à **faux**, l'exécution du programme **s'arrête**.

(Fonctionnalité présente dans quasiment tous les langages.)

SYNTAXE

```
assert <expression booléenne>, "message d'erreur"
```

Sémantique : le message d'erreur est affiché si l'assertion est évaluée à faux ; sinon l'**exécution est silencieuse**.



Un test qui réussit ne dit **rien** : c'est normal, et c'est ce qui permet d'en enchaîner des dizaines. Le silence est la bonne nouvelle ; le message n'apparaît qu'en cas d'échec.

La fonction de test

CONVENTION

On regroupe tous les jeux de tests associés à une fonction donnée dans une **fonction de test**, nommée ainsi :

```
def test_<nom fonction a tester>(...):
    ...
```

Question 4 du cours : la fonction de test

PROPOSITION DE CORRECTION

```
def test_est_palindrome():
    # Cas extrêmes
    assert est_palindrome([]) == True, "palindrome : liste vide"
    assert est_palindrome([7]) == True, "palindrome : singleton"
    # Longueur impaire
    assert est_palindrome([1, 2, 1]) == True, "palindrome : impaire oui"
    assert est_palindrome([1, 2, 3]) == False, "palindrome : impaire non"
    # Longueur paire
    assert est_palindrome([1, 2, 2, 1]) == True, "palindrome : paire oui"
    assert est_palindrome([1, 2, 3, 4]) == False, "palindrome : paire non"
    # Cas issus du code (boîte blanche)
    assert est_palindrome([1, 2]) == False, "palindrome : sortie au 1er tour"
    assert est_palindrome([1, 2, 3, 1]) == False, "palindrome : sortie au 2e tour"
    print("test_est_palindrome : tous les tests sont passes")
```

On appelle ensuite `test_est_palindrome()` depuis le programme principal — et on peut décider de ne plus l'appeler une fois la mise au point terminée.

11 Compléments et pièges

diapos 30-32

Assertions et efficacité

- les instructions `assert` peuvent être **coûteuses en temps d'exécution ou en mémoire** ;
- après la phase de mise au point, on peut **désactiver toutes les instructions `assert`** en lançant python avec l'option `-O` ou `-OO` (mode optimisé) :

```
python3 -O fichier.py
```

ATTENTION

L'instruction `assert` est **exclusivement réservée au débogage**. Les assertions étant automatiquement désactivées en mode optimisé, elles **ne doivent pas intervenir dans le fonctionnement d'un programme** autrement que pour en vérifier la **correction de l'écriture**.

En particulier :

- les expressions booléennes utilisées dans les assertions **ne doivent pas avoir d'effets de bord**.
Exemple à éviter : `assert liste.append(5)!=None` — en mode optimisé, l'ajout ne se ferait plus ;
- les assertions ne doivent être utilisées **ni pour valider des entrées utilisateur, ni pour valider le contenu de fichiers**.

NE PAS CONFONDRE

L'instruction `assert` permet de vérifier une assertion **exécutable**. Elle ne doit pas être confondue avec la vérification d'**assertions formelles** (mot-clef `assert`) qui permettent de guider un outil de preuve d'algorithme — vu dans l'UE **Info2.Algo2**.

POUR ALLER PLUS LOIN

Utiliser une librairie dédiée (`unittest...`) pour :

- **structurer les tests** en modules, classes et fonctions ;
- disposer d'un **jeu d'assertions plus riche** ;
- **lancer automatiquement** les tests ;
- obtenir un **feedback d'exécution** pour chaque test.

Vu dans l'UE **Info2.ILU1**. Lecture conseillée par le cours : realpython.com/python-assert-statement.

→ Glossaire des définitions

Clean code diapo 3	Construction de logiciels en minimisant le coût du changement et en maximisant la capacité à répondre au changement.
KISS diapo 4	<i>Keep It Simple, Stupid</i> : éviter l'optimisation prématurée.
Règle du boy scout diapo 4	Laisser le code source plus propre que quand on l'a trouvé.
snake_case diapo 8	Convention de nommage Python (PEP 8) : mots en minuscules séparés par des tirets bas.
Factoriser diapo 16	Repérer des fonctions au code très ressemblant, mettre en commun les parties identiques, et transformer les différences en paramètres supplémentaires.
Jeu de test diapo 20	Spécification, pour une fonction et des paramètres donnés, de la valeur de retour attendue.
Test en boîte noire diapo 23	Méthode où l'on ne connaît pas le code : on se base sur l'énoncé, et l'on peut écrire les tests avant la fonction.
Test en boîte blanche diapo 25	Méthode où l'on connaît le code : on s'appuie dessus pour identifier les cas, donc après avoir écrit la fonction.
Jeu de tests complet diapo 25	Jeu de tests qui couvre l'intégralité des lignes d'une fonction.
Assertion diapo 28	Expression booléenne qui doit être évaluée à vrai ; si elle est évaluée à faux, l'exécution du programme s'arrête.
Fonction de test diapo 29	Fonction regroupant tous les jeux de tests d'une fonction donnée, nommée <code>test_<nom></code> .
Mode optimisé diapo 30	<code>python3 -O</code> (ou <code>-OO</code>) : désactive toutes les instructions <code>assert</code> .

→ Synthèse

Les deux parties du CM n'en font qu'une : **on écrit proprement pour pouvoir changer son code, et on le teste pour pouvoir le changer sans le casser**. La décomposition est le pont entre les deux — elle produit les fonctions que les tests savent examiner.

QUESTION	RÉPONSE DU COURS
Pourquoi bien écrire ?	Pour minimiser le coût du changement et maximiser la capacité à y répondre.
Comment nommer une fonction ?	Par ce qu'elle fait — le quoi, pas le comment — si possible avec un verbe d'action, en snake_case et en français dans cette UE.
Que doit dire un commentaire ?	Le pourquoi, pas le comment. Un bon nommage rend beaucoup de commentaires inutiles.
Quand factoriser ?	Dès qu'un copier-coller a produit du code presque identique ; les différences deviennent des paramètres.
À quoi sert un jeu de tests ?	À spécifier la valeur de retour attendue pour des paramètres donnés — avec pour objectif de mettre le code en défaut.
Boîte noire ou blanche ?	Sans le code, d'après l'énoncé, avant d'écrire la fonction / avec le code, d'après les chemins d'exécution, après l'avoir écrite.
Peut-on garder les assert en production ?	Non : ils sont réservés au débogage et disparaissent en mode optimisé — donc jamais d'effet de bord ni de validation d'entrées dedans.

LE RÉFLEXE D'EXAMEN

Quand on demande des jeux de tests, ne donnez jamais une simple liste de valeurs : donnez pour chacun **le paramètre, le retour attendu, et la caractéristique qu'il couvre** (cas extrême, longueur paire, deux itérations complètes...). C'est la justification qui est notée, pas la valeur.